

Data Science

[End Sem Presentation]

DSc_grp99

Aryan Singla - 2022112

Harsh Rajput - 2022201

Niteen Kumar - 2022336

Shubham - 2022448



INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY
DELHI



KDD Cup 1999 Data



The KDD Cup 1999 dataset is a benchmark for network intrusion detection systems, widely used in research and development of anomaly detection algorithms. Originally, It consists of [41 features](#) and [23 classes](#), with 4,898,431 instances

Each connection is labeled as either normal, or as an attack, with exactly one specific attack type.
Each connection record consists of about 100 bytes.

Each record is labeled as either:

1. **Normal:** Representing legitimate traffic.
2. **Attack:** Representing malicious traffic, which can further be categorized into 4 main types:
 - **DoS (Denial of Service):** Overwhelm the target.
 - **Probe:** Gather information about the target.
 - **R2L (Remote to Local):** Gain unauthorized access remotely.
 - **U2R (User to Root):** Gain superuser/root access locally.

Feature Description



1. Basic Features (Attributes 1–9):

These are extracted directly from TCP/IP packets and include:

- **Duration:** Length of the connection in seconds.
- **Protocol_type:** Type of protocol (e.g., TCP, UDP, ICMP).
- **Service:** Network service used by the connection (e.g., HTTP, FTP, SMTP).
- **Flag:** Status of the connection (e.g., SF for successful connection).
- **Src_bytes:** Number of bytes sent from the source to the destination.
- **Dst_bytes:** Number of bytes sent from the destination to the source.
- **Land:** Boolean indicating whether the source and destination addresses/ports are the same.
- **Wrong_fragment:** Number of wrong fragments.
- **Urgent:** Number of urgent packets.

2. Content Features (Attributes 10–22):

These focus on the payload data within packets and include:

- **Hot:** Number of "hot" indicators (e.g., shell prompt).
- **Failed_logins:** Number of failed login attempts.
- **Root_shell:** Boolean indicating root shell access.
- **Su_attempted:** Number of "su root" attempts.
- **Num_file_creations:** Number of file creation operations.
- **Num_access_files:** Number of file access operations.
- **Num_outbound_cmds:** Number of outbound commands in an FTP session.
- **Is_host_login:** Boolean for login as root user.
- **Is_guest_login:** Boolean for login as a guest user.

Feature Description



3. Traffic Features (Attributes 23–31):

These are aggregated over a window of time and connections:

- **Count:** Number of connections to the same host in the last 2 seconds.
- **Srv_count:** Number of connections to the same service in the last 2 seconds.
- **Serror_rate:** Percentage of connections with SYN errors.
- **Srv_serror_rate:** Percentage of connections with SYN errors to the same service.
- **Rerror_rate:** Percentage of connections with REJ errors.
- **Srv_rerror_rate:** Percentage of connections with REJ errors to the same service.
- **Same_srv_rate:** Percentage of connections to the same service.
- **Diff_srv_rate:** Percentage of connections to different services.

4. Host Features (Attributes 32–41):

These capture statistics about network hosts:

- **Dst_host_count:** Number of connections to the same destination host.
- **Dst_host_srv_count:** Number of connections to the same service as the destination host.
- **Dst_host_same_srv_rate:** Percentage of connections to the same service for the destination host.
- **Dst_host_diff_srv_rate:** Percentage of connections to different services for the destination host.
- **Dst_host_same_src_port_rate:** Percentage of connections with the same source port to the destination host.
- **Dst_host_srv_diff_host_rate:** Percentage of different hosts for the same service to the destination host.
- **Dst_host_serror_rate:** Percentage of connections with SYN errors for the destination host.
- **Dst_host_srv_serror_rate:** Percentage of connections with SYN errors to the same service for the destination host.

Problem Statement



The KDD Cup 1999 dataset is a widely used benchmark for research in intrusion detection systems (IDS), providing realistic and comprehensive network traffic data that includes both normal and malicious activities. Created for the Knowledge Discovery and Data Mining Tools Competition, this dataset challenges us to develop algorithms capable of distinguishing between benign and anomalous traffic, simulating a range of cyber threats such as denial-of-service attacks, probing, remote-to-local, and user-to-root attacks.

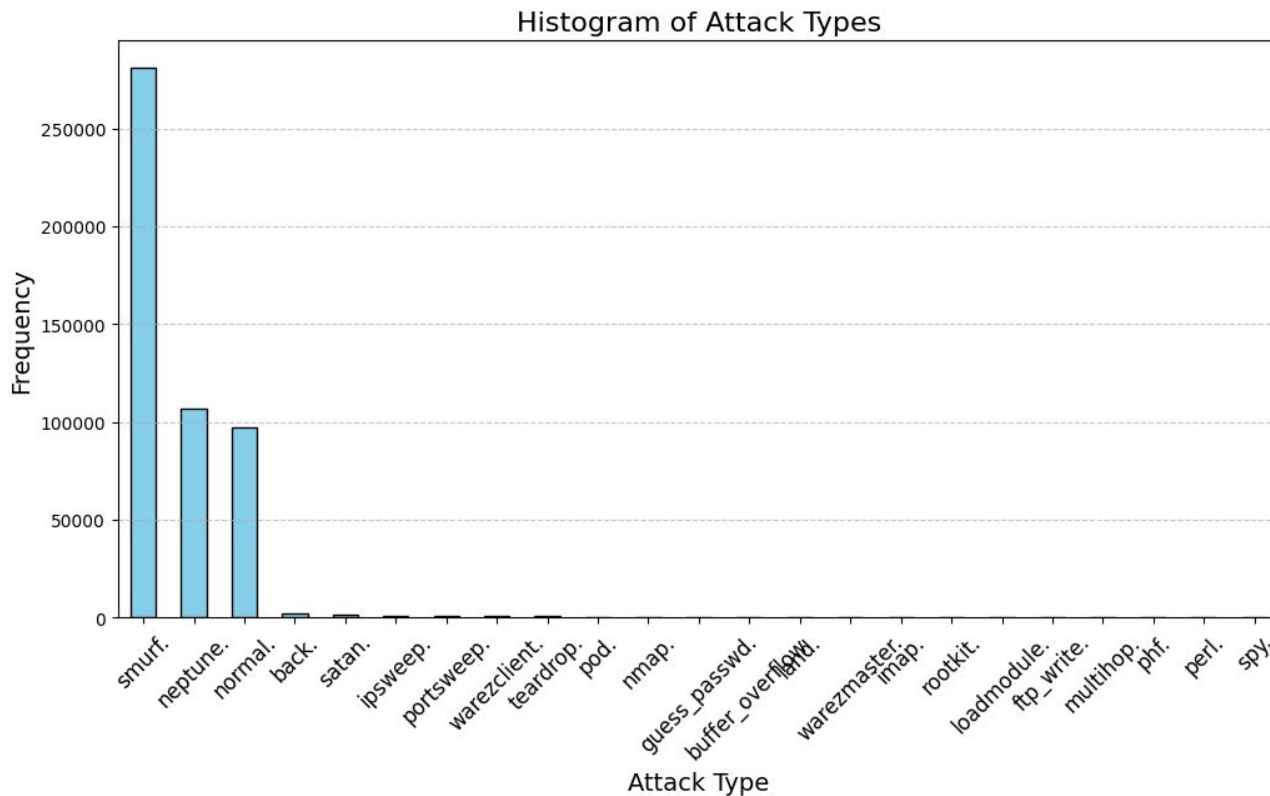
Significance

It is widely used to evaluate machine learning models in handling imbalanced data, as 80% of the records represent attacks. Accurate classification can enable real-time detection of threats, reducing the cost of security breaches, which averaged \$4.45 million globally in 2023. Moreover, the dataset's complexity, fosters research in feature engineering and algorithm scalability, making it valuable for both academic and industrial applications.

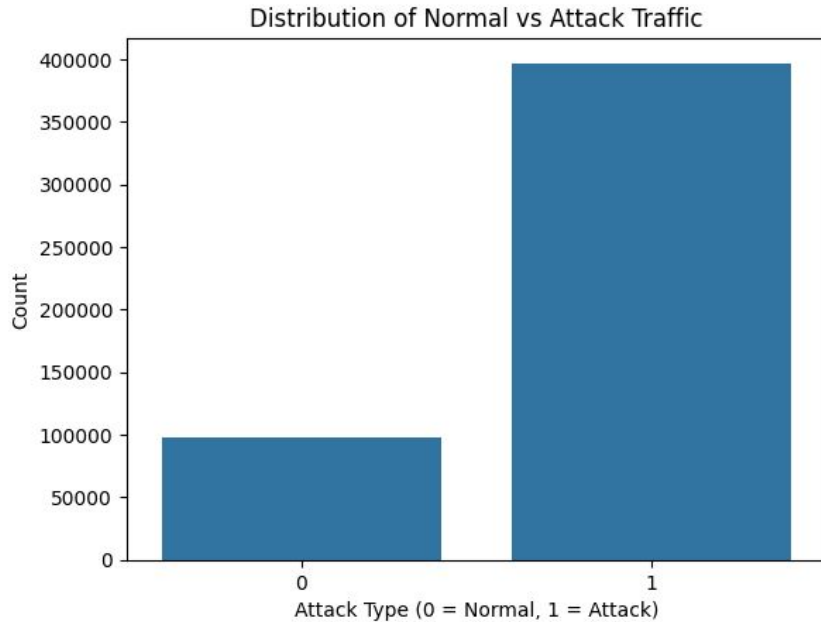
EDA - Attack Type



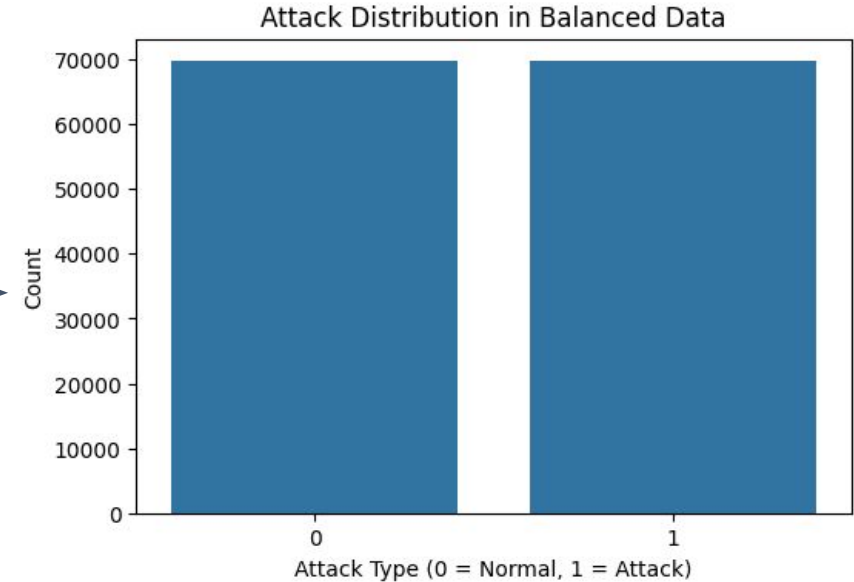
attack_type	
smurf.	280790
neptune.	107201
normal.	97278
back.	2203
satan.	1589
ipsweep.	1247
portsweep.	1040
warezclient.	1020
teardrop.	979
pod.	264
nmap.	231
guess_passwd.	53
buffer_overflow.	30
land.	21
warezmaster.	20
imap.	12
rootkit.	10
loadmodule.	9
ftp_write.	8
multihop.	7
phf.	4
perl.	3
spy.	2



Class Distribution

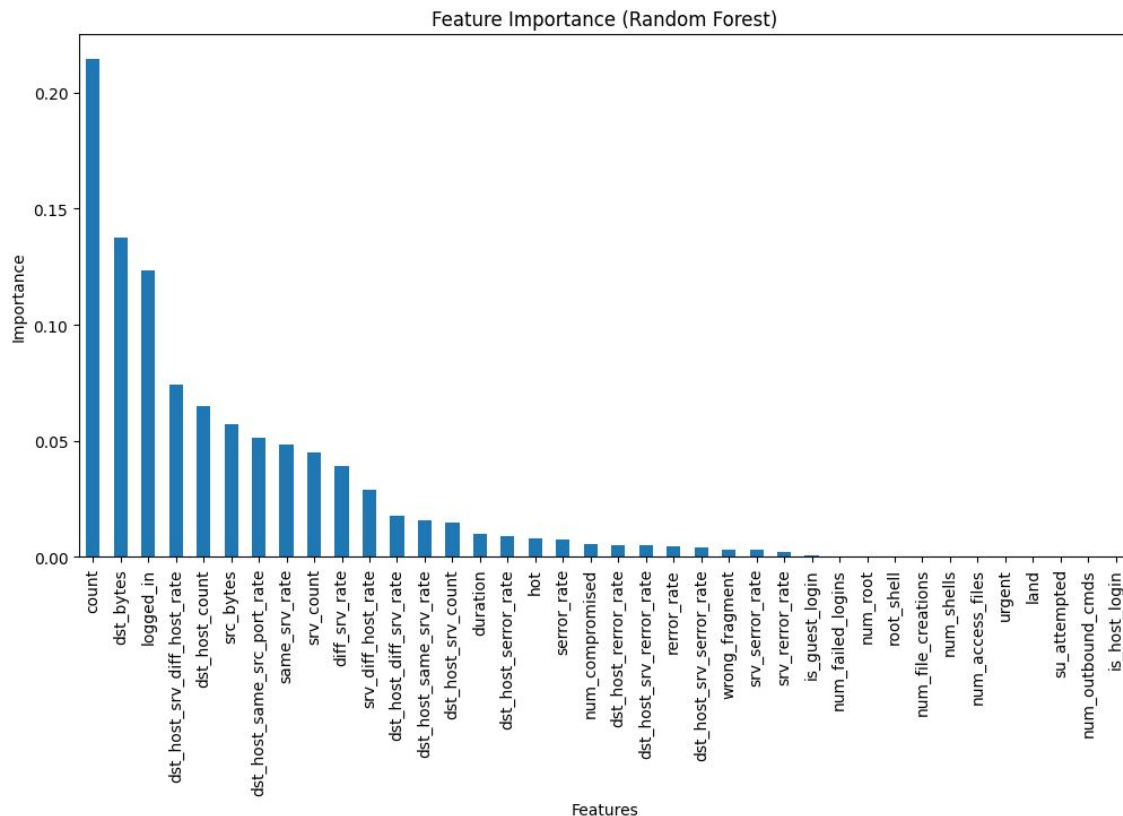


Classes were imbalanced



Balanced using oversampling (SMOTE)

Top Features Correlated with target



The top 10 most important features, as identified by the Random Forest model, include:

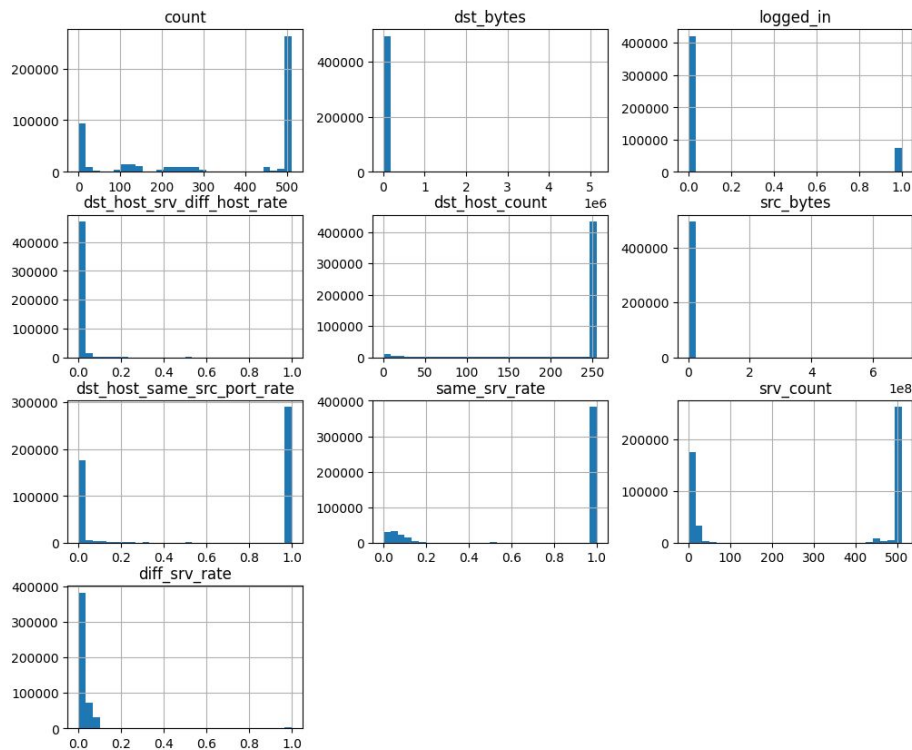
- **count, dst_bytes, logged_in, dst_host_srv_diff_host_rate, dst_host_count, src_bytes, dst_host_same_src_port_rate, same_srv_rate, srv_count, diff_srv_rate.**

These features play a significant role in distinguishing between normal and attack traffic.

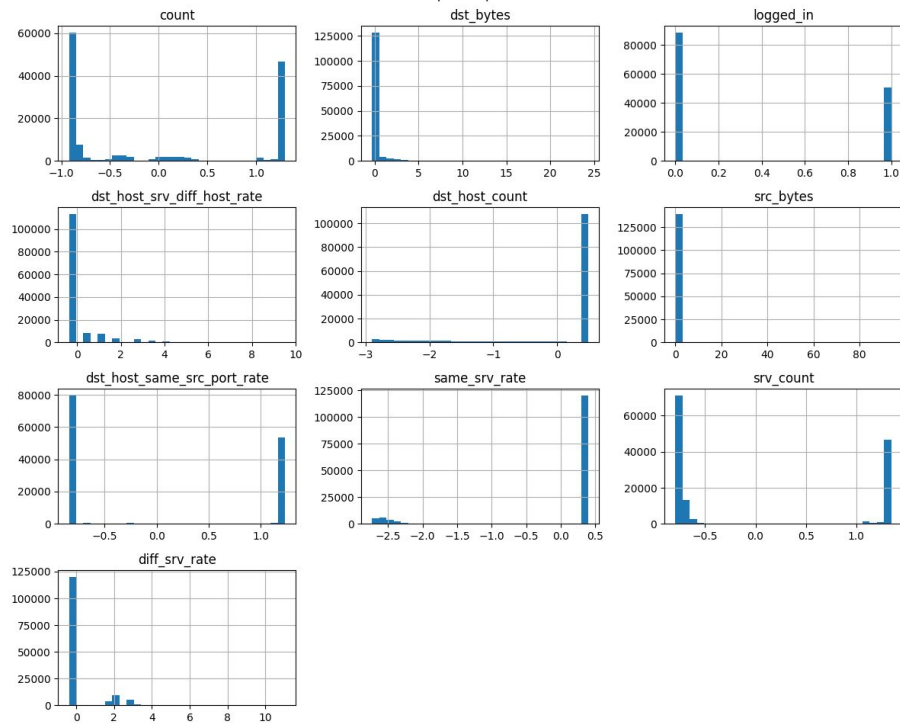
Feature Distribution



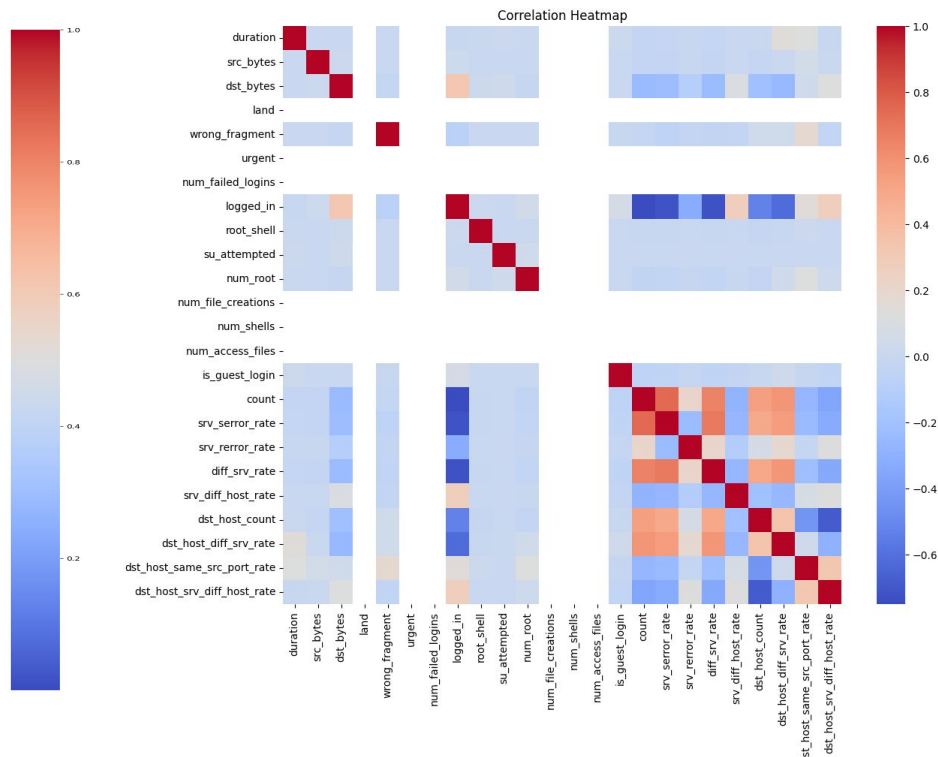
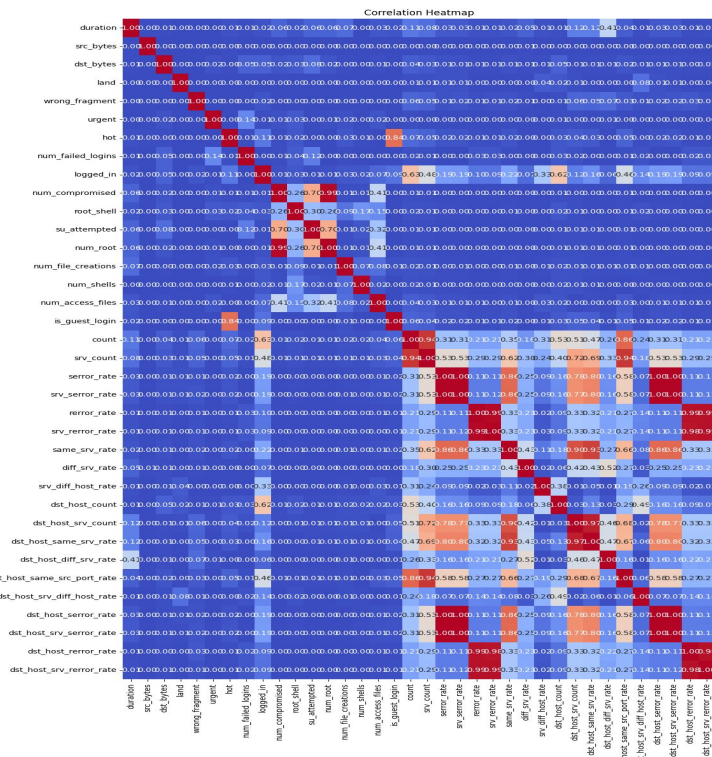
Distributions of Top 10 Important Features



Distributions of Top 10 Important Features



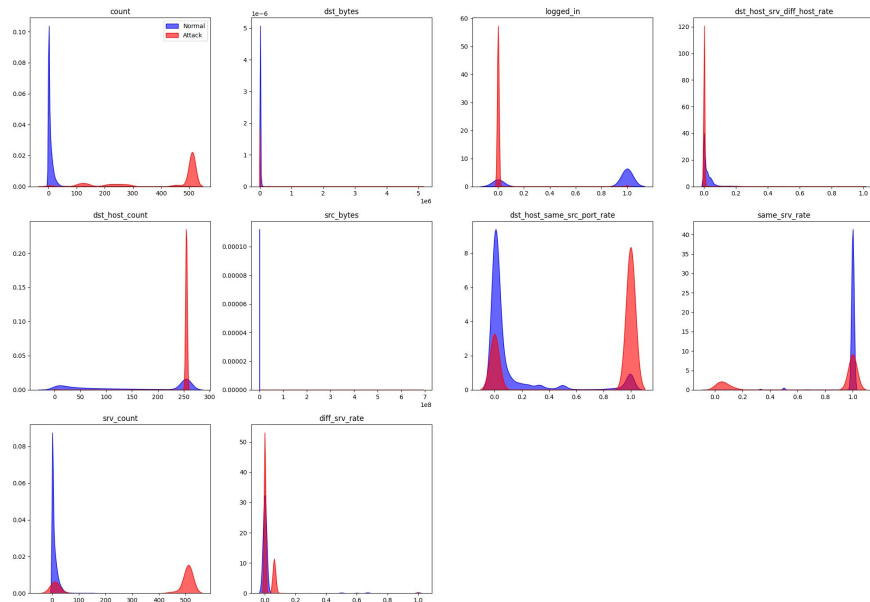
Correlation Matrix



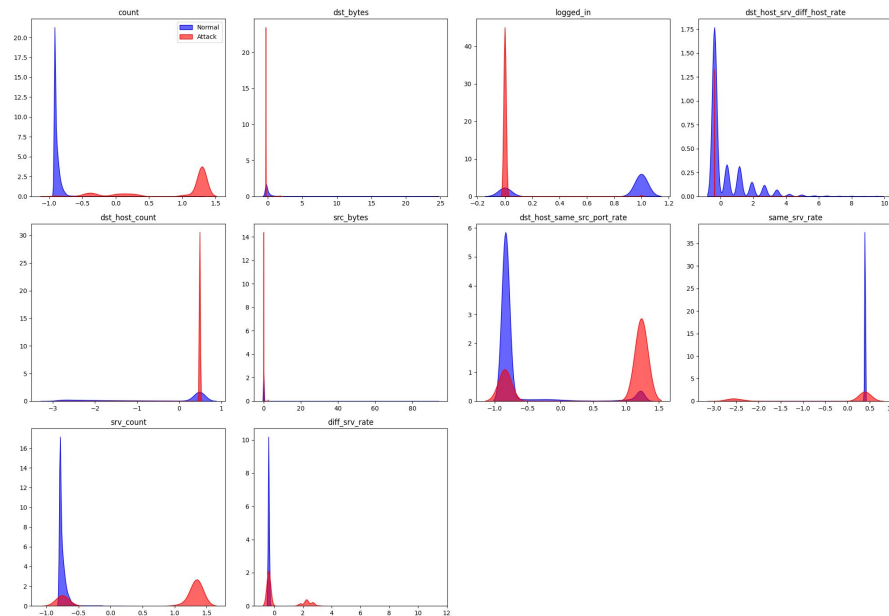
Feature Distribution With Target



KDE Plots of Top 10 Features for Normal vs Attack Traffic



KDE Plots of Top 10 Features for Normal vs Attack Traffic



Outlier Detection and Removal



Method: Outliers are identified as data points with a z-score (standardized value) exceeding a threshold (>3 or <-3).

```
Number of Outliers Detected for Each Feature:
count                                0
dst_bytes                           178
logged_in                           0
dst_host_srv_diff_host_rate         5567
dst_host_count                       26977
src_bytes                            60
dst_host_same_src_port_rate          0
same_srv_rate                        0
srv_count                            0
diff_srv_rate                        4679
dtype: int64
Total rows with outliers in at least one feature: 32320
```

Action: Rows with outliers in any of the top 10 features were Identified and Carefully evaluated

Results After Preprocessing



- **Balanced Dataset:** The dataset is now balanced with an equal number of normal and attack traffic instances.
- **Cleaned Data:** The outlier removal step reduced the size of the dataset by eliminating rows with extreme values in the top 10 features.
- **Standardized Data:** All non-binary features were scaled, making the data ready for modeling.

This preprocessed dataset is now ready for training machine learning models, such as classification algorithms, that can effectively distinguish between normal and attack traffic.

Hypothesis Testing



The goal of this analysis is to determine if there are statistically significant relationships between certain network traffic features and whether the traffic is normal or an attack.

- **Chi-Square Test:** Testing for the association between categorical features and the attack type.
- **T-test:** Comparing the means of numerical features between normal traffic and attack traffic.

Sample Size: random sampling step.

- We randomly selected 5000 samples from the dataset for testing to ensure representative results.

Significance Level:

- We used a significance level of 0.05 for all tests, meaning that p-values below 0.05 indicate a significant relationship.

Hypothesis Testing – chi-square test



Feature	Statistic (Sampled Data)	p-value (Sampled Data)	Significance (Sampled Data)	Chi-Square Statistic (Full Dataset)	p-value (Full Dataset)	Significance (Full Dataset)
Protocol Type (TCP)	72.87	1.38×10^{-17}	Significant	2281.20	0.0	Significant
Protocol Type (UDP)	71.87	2.30×10^{-17}	Significant	2214.37	0.0	Significant
Service (FTP)	1.43	0.23	Not Significant	25.50	4.43×10^{-7}	Significant
Service (HTTP)	2945.80	0.0	Significant	73932.58	0.0	Significant
Flag Type (SF)	4152.08	0.0	Significant	106545.26	0.0	Significant



Hypothesis Testing – T test



Feature	T-Statistic (Sampled Data)	p-value (Sampled Data)	Significance (Sampled Data)	T-Statistic (Full Dataset)	p-value (Full Dataset)	Significance (Full Dataset)
Duration	2.30	0.0215	Significant	11.76	6.72×10^{-32}	Significant
src_bytes	-0.57	0.5705	Not Significant	2.48	0.0132	Significant
dst_bytes	23.39	2.11×10^{-111}	Significant	110.78	0.0	Significant
count	-96.99	0.0	Significant	-507.54	0.0	Significant
srv_count	1.68	0.0933	Not Significant	7.23	4.78×10^{-13}	Significant

ML model – Logistic regression



Description: Logistic Regression is a linear model used for binary classification. It estimates the probability of the default class (e.g., `attack_type = 1`) using a logistic (sigmoid) function.

MaxIter = 1000

Pros: Simple, interpretable, and fast for small to medium-sized datasets.

Cons: Assumes a linear decision boundary, which might not capture complex patterns in the data.

```
Training Accuracy: 0.9966
Test Accuracy: 0.9963

Classification Report (Training Data):
              precision    recall  f1-score   support

     0       0.99         0.99         0.99        77925
     1       1.00         1.00         1.00       317291

   accuracy                1.00        395216
  macro avg       0.99         1.00         0.99        395216
 weighted avg     1.00         1.00         1.00        395216

Classification Report (Testing Data):
              precision    recall  f1-score   support

     0       0.99         0.99         0.99        19353
     1       1.00         1.00         1.00       79452

   accuracy                1.00        98805
  macro avg       0.99         1.00         0.99        98805
 weighted avg     1.00         1.00         1.00        98805

Confusion Matrix (Testing Data):
[[19222  131]
 [ 230 79222]]
```

ML model – XGBoost



Description: Gradient Boosting is an ensemble technique that builds multiple decision trees sequentially. Each tree tries to correct the errors of the previous one. XGBoost is an optimized version of gradient boosting.

Pros: Highly accurate, handles non-linearity, works well with imbalanced datasets, can handle complex data patterns.

Cons: Computationally expensive, can be prone to overfitting if not tuned properly.

Training Accuracy: 1.0000

Test Accuracy: 0.9999

Classification Report (Training Data):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	77925
1	1.00	1.00	1.00	317291
accuracy			1.00	395216
macro avg	1.00	1.00	1.00	395216
weighted avg	1.00	1.00	1.00	395216

Classification Report (Testing Data):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19353
1	1.00	1.00	1.00	79452
accuracy			1.00	98805
macro avg	1.00	1.00	1.00	98805
weighted avg	1.00	1.00	1.00	98805

Confusion Matrix (Testing Data):

```
[[19346   7]
 [   7 79445]]
```

Scaling Techniques – SVD using PCA



10.13.1 Principal component analysis (PCA)

The basic approach in principal components or *Karhunen-Loève transform* is conceptually quite simple. First, the d -dimensional mean vector μ and $d \times d$ covariance matrix Σ are computed for the full data set. Next, the eigenvectors and eigenvalues are computed (cf. Appendix ??), and sorted according to decreasing eigenvalue. Call these eigenvectors e_1 with eigenvalue λ_1 , e_2 with eigenvalue λ_2 , and so on. Next, the largest k such eigenvectors are chosen. In practice, this is done by looking at a spectrum of eigenvectors. Often there will be xxx implying an inherent dimensionality of the subspace governing the “signal.” The other dimensions are noise. Form a $k \times k$ matrix A whose columns consist of the k eigenvectors.

Preprocess data according to:

$$\mathbf{x}' = A^t(\mathbf{x} - \mu). \quad (90)$$

While PCA uses SVD in its computation. SVD is a more general matrix factorization technique, whereas PCA is a specific statistical method used for dimensionality reduction.

We transform the data into a lower dimension using SVD by reducing it to its top k singular components, where $k = 10$ is the reduced number of dimensions.

```
Logistic Regression - Training Accuracy: 0.9852
Logistic Regression - Test Accuracy: 0.9843

Logistic Regression - Classification Report (Test Data):
              precision    recall  f1-score   support

     0           0.95         0.97         0.96         19353
     1           0.99         0.99         0.99         79452

 accuracy          0.98
macro avg          0.98         0.98         0.98         98805
weighted avg          0.98         0.98         0.98         98805

Logistic Regression - Confusion Matrix (Test Data):
[[18863  490]
 [ 1062 78390]]
```

```
XGBoost - Training Accuracy: 0.9998
XGBoost - Test Accuracy: 0.9993

XGBoost - Classification Report (Test Data):
              precision    recall  f1-score   support

     0           1.00         1.00         1.00         19353
     1           1.00         1.00         1.00         79452

 accuracy          1.00
macro avg          1.00         1.00         1.00         98805
weighted avg          1.00         1.00         1.00         98805

XGBoost - Confusion Matrix (Test Data):
[[19322   31]
 [   43 79409]]
```

Scaling Techniques – JL Lemma



We transform the data into a lower dimension with the help of a JL matrix (whose entries are sampled from $N(0, 1/k)$, where k is the size of the reduced number of dimensions, in our case, $n_components=10$). The JL lemma ensures that distance between points is maintained even after transformation, a useful property that maintains structure. Thus, we utilize this technique for our case, since the classification could rely on these patterns in the data. We also try out a sparse JL matrix to reduce computation time.

There are **two common types** of JL reductions:

1. **Regular JL Lemma (Gaussian Random Projection):** This technique projects the data using a Gaussian distribution.
2. **Sparse JL Lemma (Sparse Random Projection):** This technique uses a sparse matrix (with many zeros) to project the data (density = $1 / \text{root}(n)$), offering more efficiency and sparsity.

Scaling Techniques – JL Lemma



Logistic Regression (Regular JL) - Training Accuracy: 0.9846
Logistic Regression (Regular JL) - Test Accuracy: 0.9843

Logistic Regression (Regular JL) - Classification Report (Test Data):

	precision	recall	f1-score	support
0	0.94	0.98	0.96	19353
1	0.99	0.99	0.99	79452
accuracy			0.98	98805
macro avg	0.97	0.98	0.98	98805
weighted avg	0.98	0.98	0.98	98805

XGBoost (Regular JL) - Training Accuracy: 0.9997
XGBoost (Regular JL) - Test Accuracy: 0.9991

XGBoost (Regular JL) - Classification Report (Test Data):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19353
1	1.00	1.00	1.00	79452
accuracy			1.00	98805
macro avg	1.00	1.00	1.00	98805
weighted avg	1.00	1.00	1.00	98805

Logistic Regression (Sparse JL) - Training Accuracy: 0.9777
Logistic Regression (Sparse JL) - Test Accuracy: 0.9779

Logistic Regression (Sparse JL) - Classification Report (Test Data):

	precision	recall	f1-score	support
0	0.96	0.93	0.94	19353
1	0.98	0.99	0.99	79452
accuracy			0.98	98805
macro avg	0.97	0.96	0.96	98805
weighted avg	0.98	0.98	0.98	98805

XGBoost (Sparse JL) - Training Accuracy: 1.0000
XGBoost (Sparse JL) - Test Accuracy: 0.9997

XGBoost (Sparse JL) - Classification Report (Test Data):

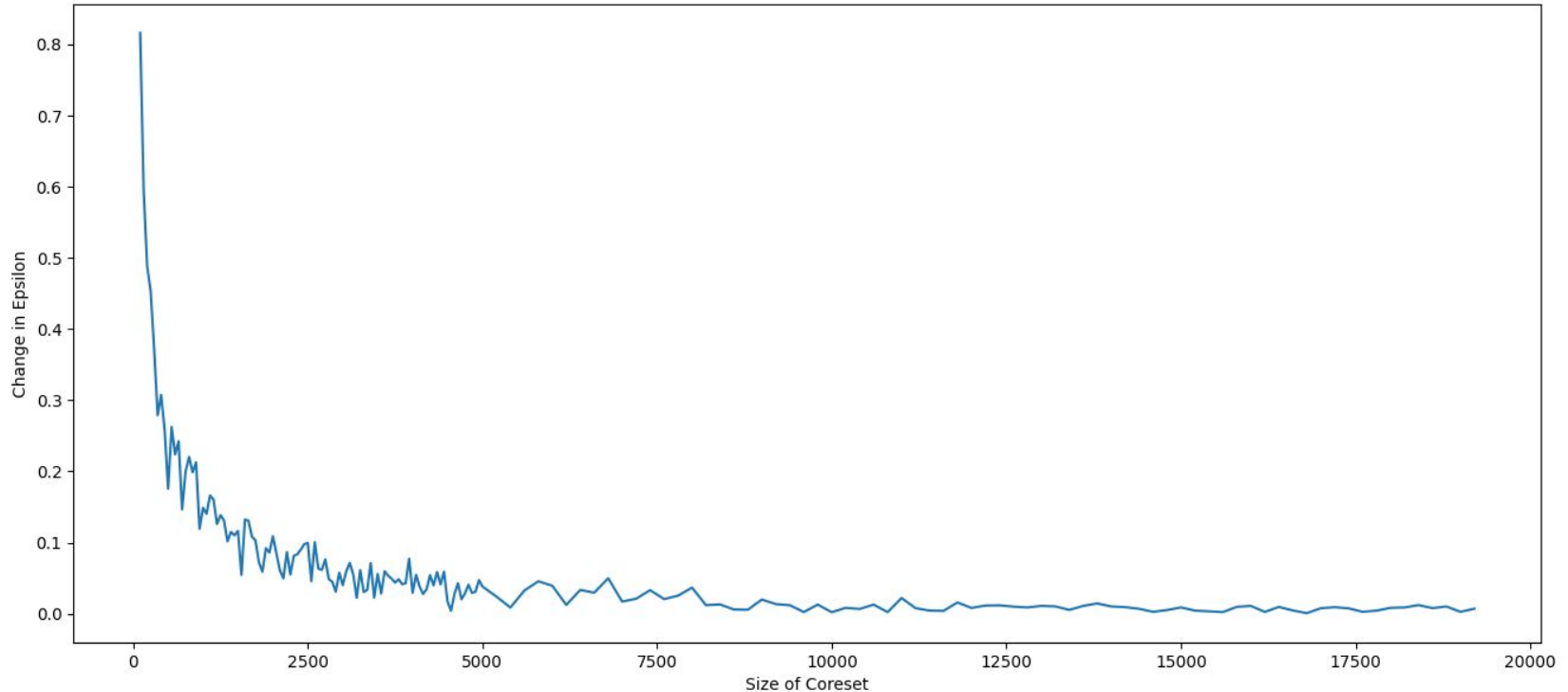
	precision	recall	f1-score	support
0	1.00	1.00	1.00	19353
1	1.00	1.00	1.00	79452
accuracy			1.00	98805
macro avg	1.00	1.00	1.00	98805
weighted avg	1.00	1.00	1.00	98805

Scaling Techniques – Coresets(K Means)

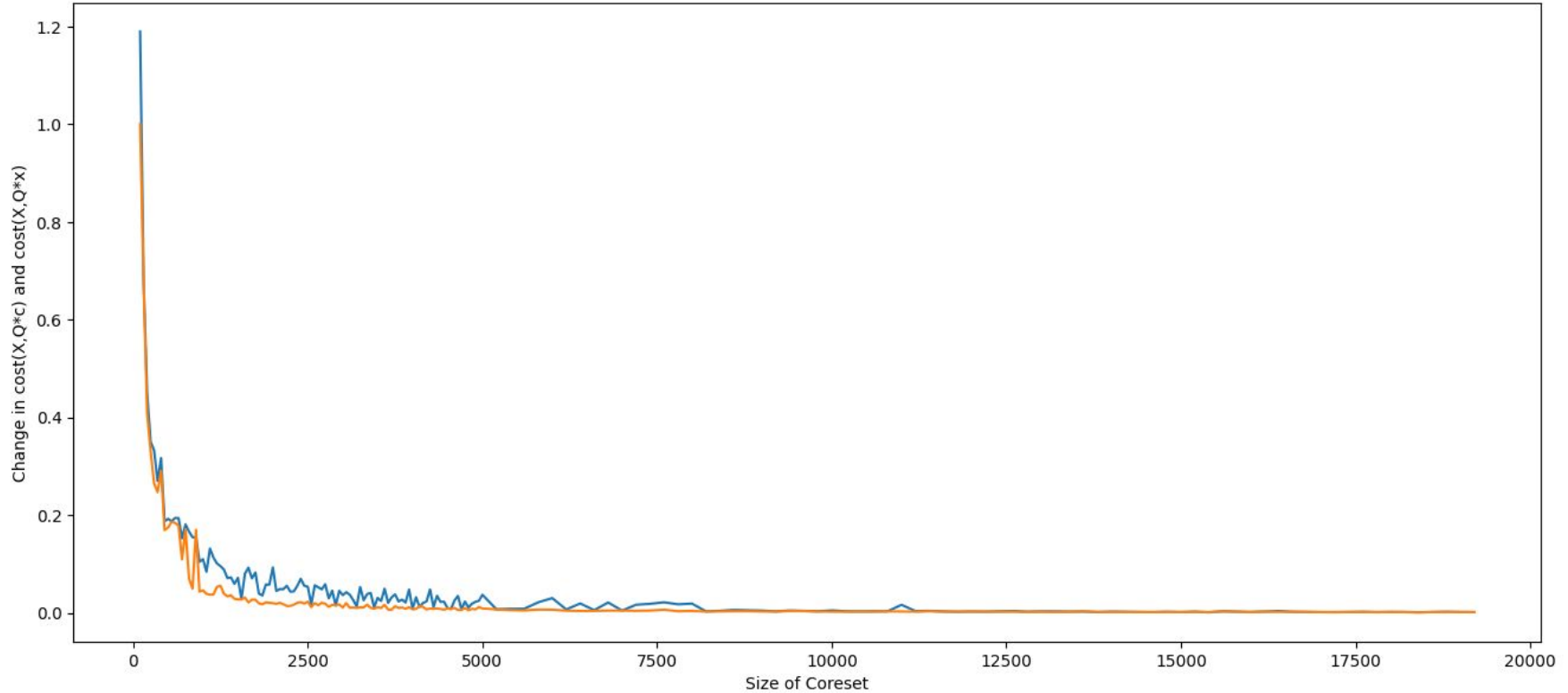
We introduced coresets as small, weighted summaries of large data sets such that solutions found on the summary itself are provably competitive with solutions found on the full data set in terms of a specific task, like classification or clustering. In our approach, we first experiment with various coreset sizes, and after iterating, we determine that the best coreset size and it is 5,000 (1% of original size). This coreset was created by D2 sampling of the data points, ensuring that the selected subset retains the key information needed for the task. Using the coreset helps improve efficiency without sacrificing the quality of the model's performance.

```
sum of weights of X : 494021.0
sum of weights of C : 489069.5046560087
cost(X,Q*c) : 2209250.255535879
cost(C,Q*c) : 2153293.030875129
cost(X,Q*x) : 2203843.0870299623
cost(C,Q*x) : 2160770.6739207683
```

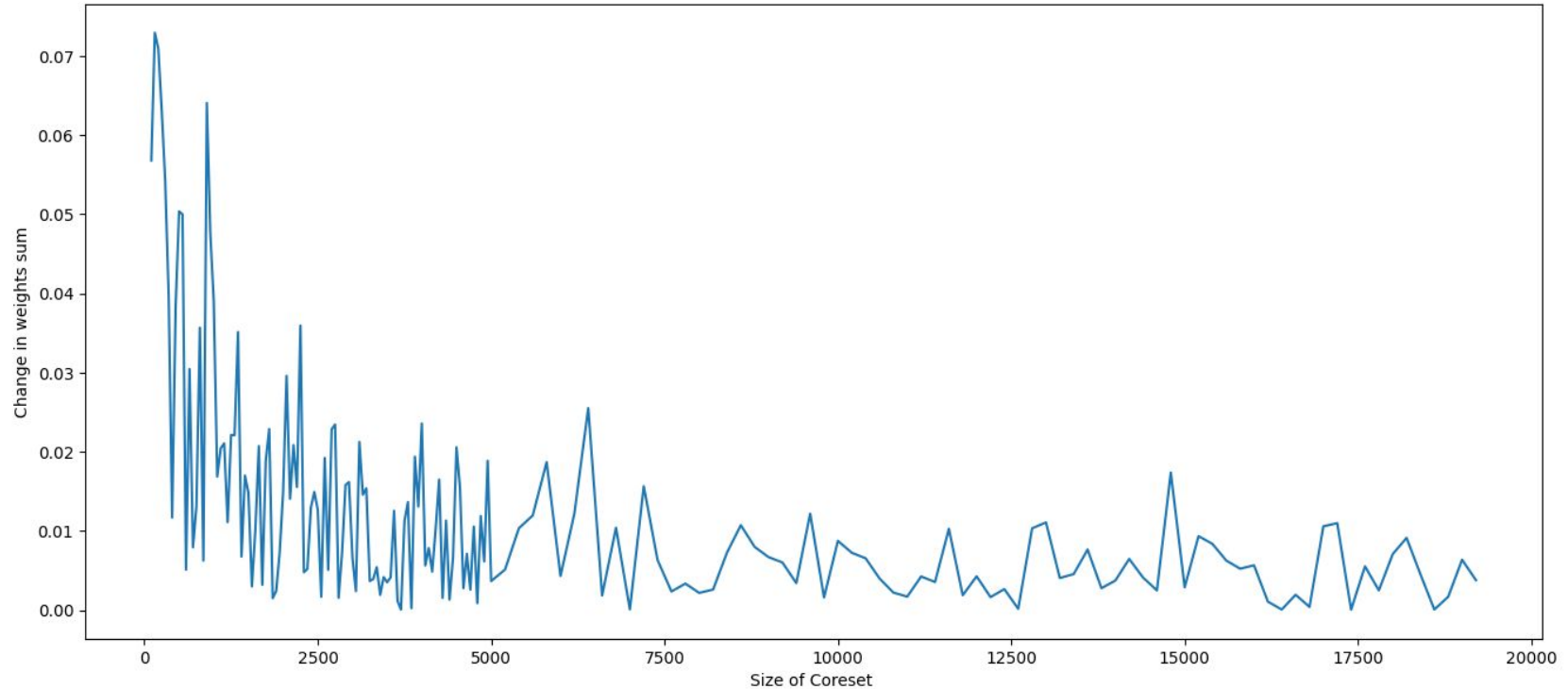
Scaling Techniques – Coresets



Scaling Techniques – Coresets



Scaling Techniques – Coresets



Conclusion and Future Work



Conclusion

- Successfully conducted **Exploratory Data Analysis (EDA)** to understand the KDD Cup 1999 dataset, revealing key patterns and insights in network traffic data.
- Performed **hypothesis testing** to validate assumptions about feature significance, aiding feature selection and data preprocessing.
- Trained an initial **Logistic Regression, XGBoost model**, achieving promising results with high accuracy and efficient runtime, demonstrating its potential for intrusion detection.
- **SVD and JL Scaling** Reduced the classification accuracy by a small margin $< 2\%$, and **SVD perform Slightly Better than JL Scaling**
- **Coresets for K Means** worked very well, achieving **epsilon of less than < 0.01**

Future Work

1. **Tune hyperparameters to further improve detection accuracy and reduce false-positive rates.**
2. **Experimenting with more advanced ML models, DL based methods for scaling (autoencoders and more).**

Thank You